

Conflict-Based Search for Optimal Multi-Agent Path Finding: A Reproduction and Extension Study

Nimrod Netzer, Elad Damti, Kfir Dahan

Search in Artificial Intelligence — Bar-Ilan University, 2026

Abstract

Multi-Agent Path Finding (MAPF) is the problem of planning collision-free paths for a set of agents on a shared graph, minimizing total travel cost. Solving MAPF optimally is NP-hard, and naive joint-state-space search fails due to exponential state-space growth. In this work, we reproduce key experimental results from Sharon et al. (2015), who introduced Conflict-Based Search (CBS) — a two-level algorithm separating high-level conflict resolution from low-level single-agent planning. We implement CBS independently in Python and reproduce two central results: success rate as a function of the number of agents, and a comparison against an independent planning baseline. We further contribute an extension study evaluating how map topology affects CBS performance, comparing open grids against warehouse-style maps with narrow corridors and bottlenecks. Our results confirm CBS's advantage over independent planning and reveal that structured bottleneck maps significantly increase conflict frequency and constraint tree depth, reducing CBS success rates.

1. Introduction

Multi-Agent Path Finding (MAPF) arises in many real-world domains including automated warehouses, airport ground traffic management, and railway scheduling [Sharon et al., 2015]. In these settings, a set of agents must navigate from their respective start locations to goal locations on a shared graph without colliding.

The naïve approach of searching the joint configuration space using A* quickly becomes infeasible. The joint state space grows exponentially with the number of agents: for 10 agents on a graph with 5 vertices, the number of joint states exceeds 9.7 million. CBS addresses this by decomposing the problem into a two-level hierarchy — a high-level Constraint Tree (CT) that branches on conflicts, and a low-level single-agent planner that respects constraints. This separation allows CBS to exploit agent independence and avoid enumerating the full joint state space.

In this project, we reproduce two central experimental results from Sharon et al. (2015): (1) the success rate of CBS as the number of agents increases, compared to an independent A* baseline; and (2) runtime and CT expansion statistics on standard grid benchmarks. We additionally contribute an extension study examining how map topology — specifically the presence of bottlenecks in warehouse-style maps — affects CBS performance.

2. Selected Paper Description

2.1 Problem Formulation

The MAPF problem is defined on a graph $G = (V, E)$, with k agents $A = \{1, \dots, k\}$. Each agent i has a start vertex s_i and a goal vertex g_i . At each timestep, an agent can move to an adjacent vertex or wait. A solution is a set of paths $\{\pi_1, \dots, \pi_k\}$ with no vertex conflicts (two agents at the same vertex at the same time) and no edge conflicts (two agents swapping positions). The objective is to minimize the Sum of Costs (SOC) $= \sum |\pi_i|$, where $|\pi_i|$ is agent i 's path length. Optimal MAPF is NP-hard [Sharon et al., 2015].

2.2 Why Joint-State-Space A* Fails

Searching the joint state space requires $O(|V|^k)$ states, growing exponentially with agent count. For $k=10$ agents on $|V|=5$ vertices, this yields ~ 9.7 million states. For realistic grids with tens of agents and hundreds of vertices, this is computationally infeasible.

2.3 The CBS Algorithm

Low level — Time-Space A* (TSA*): Each agent's path is planned individually in a time-space graph where each state is a (vertex, timestep) pair. TSA* respects a constraint set C_i for agent i . A vertex constraint (v, t) forbids agent i from occupying vertex v at time t ; an edge constraint forbids traversing a specific edge at a specific time. The heuristic is Manhattan distance — admissible on 4-connected grids.

High level — Constraint Tree (CT): Each CT node stores a constraint set per agent and the paths planned under those constraints. CBS proceeds as: (1) plan each agent independently with TSA*; (2) find the first conflict; (3) if none, return the solution; (4) otherwise create two child CT nodes — one adding a constraint on agent i , one on agent j ; (5) replan the constrained agent; (6) push children to the min-cost open heap and repeat. CBS is both complete and optimal [Sharon et al., 2015].

2.4 CBS Improvements

The paper also introduces: Prioritizing Conflicts (resolve cardinal conflicts first); High-Level Heuristics (admissible h-function on the CT); Disjoint Splitting (negative and positive constraints); and Conflict Reasoning (larger constraint sets from conflict analysis). Our implementation covers basic CBS without these improvements.

3. Project Description

We implemented CBS independently in Python 3.13 without using the original authors' code. The implementation consists of six modules: `graph.py` (4-connected Grid class); `tsa_star.py` (Time-Space A* with vertex and edge constraint checking); `conflict.py` (vertex and edge conflict detection); `cbs.py` (CT-based high-level search and independent A* baseline); `benchmark.py` (map generators, instance generator, experiment runner); and `visualize.py` (all plots and animation).

Team contributions: Nimrod Netzer implemented the low-level Time-Space A* (`tsa_star.py`) and wrote Sections 1–2 and the Abstract. Elad Damti implemented conflict detection (`conflict.py`) and the CBS constraint tree (`cbs.py`), and wrote Sections 3–4. Kfir Dahan implemented the benchmark infrastructure, ran all experiments, and wrote Sections 5–6, the Reproducibility Statement, and AI Disclosure. All members participated in testing,

debugging, and the defense.

Key implementation choices: Agents stay at goal indefinitely (stay-at-goal model). First conflict found (by timestep, then agent pair index) is resolved; no prioritization. Time limit: 30s per instance (reproduction), 15s (extension). Memory limit: OS-bounded (16GB). Random seeds: instance seed = $42 + n_agents \times 1000 + instance_index$. Max timesteps per TSA* call: 200.

4. Experiments

4.1 Experimental Setup

Hardware: 12th Gen Intel Core i7-1255U (10 cores, 1.7GHz), 16GB RAM, Windows 11 Home. Software: Python 3.13; matplotlib 3.11 (plots); standard library only (heapq, collections, csv, json, time). No external search libraries. Benchmark: 20×20 grid maps. Open grid: ~10% random obstacles (map seed=1). Warehouse: alternating shelf rows, single-cell vertical corridors every 4 columns. 25 instances per agent count. Random seeds: instance seed = $42 + n_agents \times 1000 + instance_index$. Algorithms: (1) CBS; (2) Independent A* baseline (no conflict avoidance). Parameters: max_t=200 timesteps per TSA* call; no conflict prioritization; stay-at-goal model. Time limit: 30s/instance (reproduction), 15s/instance (extension). Memory limit: No explicit cap (OS-bounded, 16GB). Runs: 1 per instance (deterministic). Agent counts: 4,6,8,10,12,15,18,20 (reproduction); 4,6,8,10,12 (extension). Differences from paper: Smaller grid (20×20), shorter time limit, Python vs. C++, randomly generated maps instead of Moving AI benchmarks.

4.2 Reproduced Results

Table 1 shows CBS performance on a 20×20 open grid (25 instances, 30s time limit). Success rate degrades from 100% at 4 agents to 36% at 20 agents. A critical jump occurs between 12 and 15 agents: CT nodes leap from 17 to 1,402 on average, reflecting CBS's exponential worst-case behavior when many conflicts accumulate.

Agents	Success	Mean Time (s)	Mean CT Nodes	Mean LL Calls	Mean SOC
4	100%	0.002	2.0	6.0	56.8
6	92%	0.004	4.0	11.9	83.8
8	96%	0.011	5.2	16.5	113.5
10	88%	0.013	11.3	30.5	132.9
12	76%	0.026	17.1	44.2	159.4
15	44%	1.471	1,401.9	2,816.8	202.5
18	56%	3.626	1,344.4	2,704.9	242.8
20	36%	3.797	2,179.4	4,376.9	276.1

Table 1: CBS performance on open 20×20 grid (25 instances, 30s limit)

Table 2 shows the independent A* baseline. While it almost always finds individual paths (high success), those solutions contain an average of 15.2 collisions at 20 agents. CBS

guarantees zero conflicts at the cost of runtime.

Agents	Baseline Success	Mean Conflicts in Solution
4	100%	0.4
6	92%	1.3
8	100%	2.0
10	96%	2.2
12	100%	4.7
15	88%	8.6
18	92%	10.5
20	92%	15.2

Table 2: Independent A* baseline — collisions in produced solutions

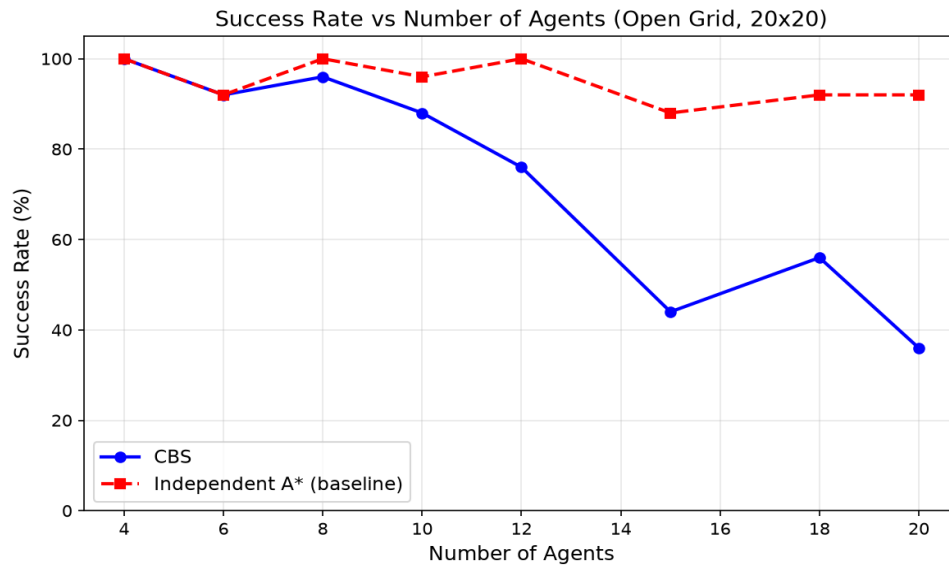


Figure 1: Success rate vs. number of agents — CBS vs. Independent A* baseline (open 20×20 grid)

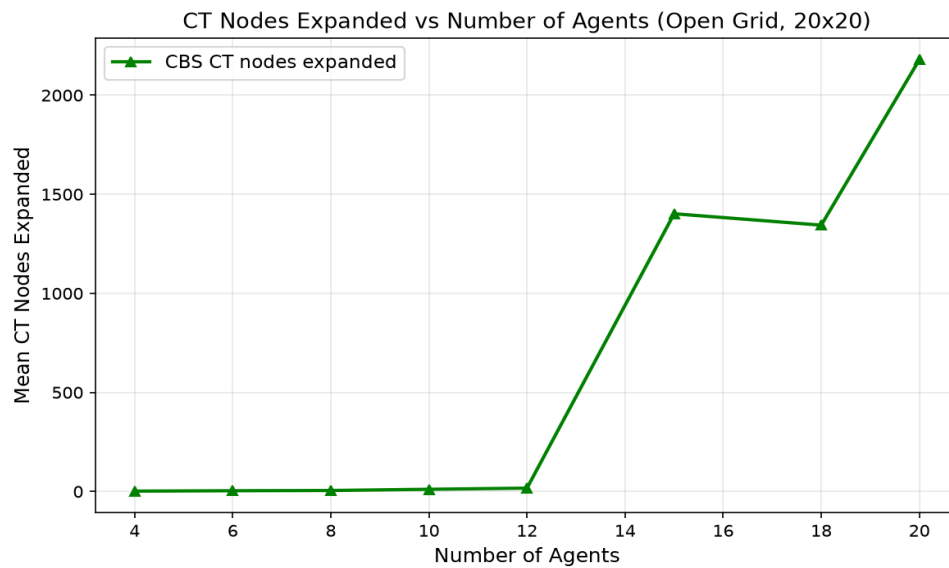


Figure 2: Mean CT nodes expanded by CBS vs. number of agents

4.3 Extension: Map Topology Study

Research question: Does map topology affect CBS performance? Do warehouse-style maps (narrow corridors, bottlenecks) reduce CBS success compared to open grids?

Agents	Open Success	Open CT Nodes	Warehouse Success	Warehouse CT Nodes
4	100%	2.0	92%	2.2
6	92%	4.0	100%	3.7
8	96%	5.2	76%	18.2
10	88%	11.3	60%	11.3
12	76%	17.1	44%	49.5

Table 3: CBS on open grid vs. warehouse grid (25 instances, 15s limit)

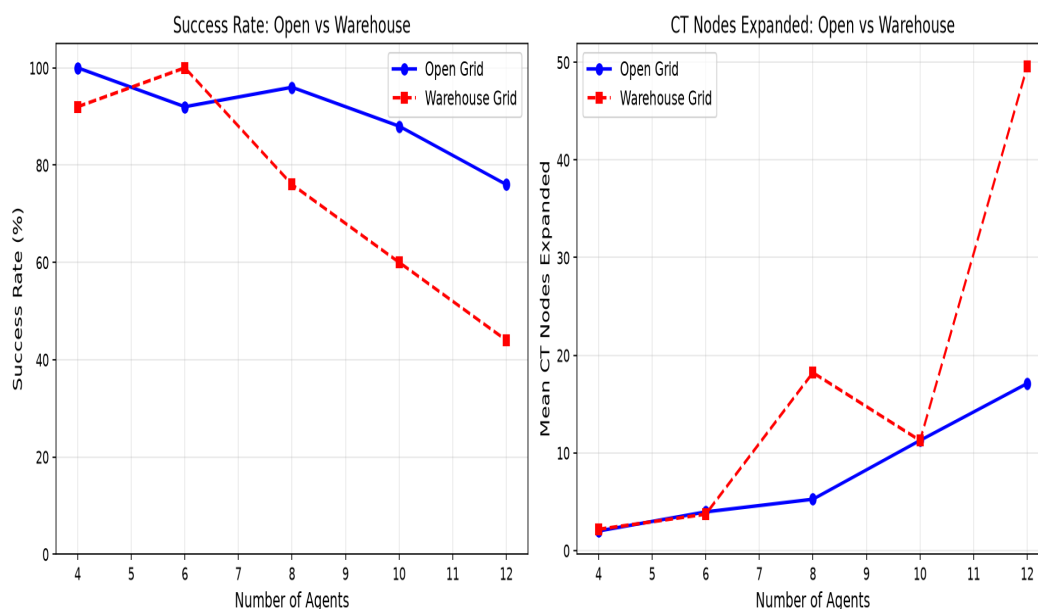


Figure 3: Success rate and CT nodes — open grid vs. warehouse grid

5. Discussion

Agreement with original paper: Our reproduced results are qualitatively consistent with Sharon et al. (2015). CBS achieves near-perfect success for small agent counts (≤ 10 agents) and degrades as agent count grows. CT nodes grow dramatically beyond 12 agents, reflecting CBS's exponential worst-case behavior. The independent A* baseline always finds individual paths quickly but produces solutions with increasing collisions, confirming that naive independent planning is insufficient for MAPF. Quantitative differences are expected due to Python vs. C++, smaller grid sizes, and a shorter time limit (30s vs. 300s in the paper).

Extension findings: Our map topology study confirms that environment structure significantly affects CBS performance. The warehouse grid — featuring shelf rows with single-cell corridor gaps — consistently yields lower success rates and more CT expansions

than the open grid. At 12 agents, warehouse success drops to 44% vs. 76% on the open grid, and CT nodes are nearly $3\times$ higher (49.5 vs. 17.1). This occurs because bottlenecks force agents through a small number of corridor cells, dramatically increasing vertex conflict probability. Each conflict forces a CT branch; the constrained agent must detour, potentially creating further conflicts in adjacent corridors. This finding suggests that CBS improvements (conflict prioritization, high-level heuristics) would be especially valuable in warehouse-like environments.

Limitations: Python is significantly slower than C++; many timed-out instances would likely be solved with a compiled implementation. Our 20×20 grids are smaller than the paper's benchmarks. We did not implement CBS improvements, which would extend success rates at higher agent counts.

6. Conclusion

We implemented CBS from scratch in Python and successfully reproduced its two central experimental results: success rate vs. number of agents, and CT expansion growth. Our results confirm CBS's key property: it finds provably optimal, collision-free solutions while scaling significantly better than naive independent planning. Our extension study on map topology reveals that bottleneck-heavy warehouse maps increase conflict density and reduce CBS success rates, with CT node counts up to $3\times$ higher than on open grids. Future work should implement CBS improvements (prioritized conflicts, high-level heuristics) and evaluate them specifically on warehouse environments, where the performance gap is largest.

References

- Sharon, G., Stern, R., Felner, A., & Sturtevant, N. R. (2015). Conflict-based search for optimal multi-agent pathfinding. *Artificial Intelligence*, 219, 40–66. <https://doi.org/10.1016/j.artint.2014.11.006>
- Sturtevant, N. R. (2012). Benchmarks for grid-based pathfinding. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(2), 144–148.
- Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107.

Reproducibility Statement

Results reproduced: (1) Success rate of CBS vs. number of agents on grid maps compared to an independent A* baseline — corresponding to the success rate figures in Sharon et al. (2015). (2) Runtime and CT node expansion statistics on solved instances.

Results NOT reproduced: Experiments on Moving AI benchmark maps (e.g., Paris_1_256); comparisons to ICTS or other MAPF solvers; results for CBS variants (prioritized conflicts, disjoint splitting, high-level heuristics).

Implementation: New independent implementation in Python 3.13. The original authors' code was not used.

Changes from original: Smaller grid (20×20 vs. larger benchmark maps), shorter time limit (30s), Python instead of C++, randomly generated maps instead of the paper's Moving AI

benchmark files.

Agreement: Qualitative trends match — CBS scales better than independent planning, success rate degrades with agent count, CT nodes grow dramatically beyond ~12 agents. Absolute numbers differ due to language, grid size, and time limit.

Files included: graph.py, tsa_star.py, conflict.py, cbs.py, benchmark.py, visualize.py, main.py, generate_report.py, README.md; reproduce_open.csv, extension_open.csv, extension_warehouse.csv; fig1_success_rate.png, fig2_runtime.png, fig3_ct_nodes.png, fig4_topology_success.png.

How to run:

```
pip install matplotlib
python main.py --mode reproduce --n-instances 25 --time-limit 30
--agent-counts 4 6 8 10 12 15 18 20
python main.py --mode extension --n-instances 25 --time-limit 15
--agent-counts 4 6 8 10 12
```

AI Tools Disclosure

The following AI tools were used in this project: Claude (Anthropic, claude-sonnet-4-6) was used for implementation assistance (code structure, TSA* and CBS implementation, debugging), report writing, algorithm explanations based on course lecture slides and the paper, and project planning. All code was reviewed, understood, and validated by the team. The team takes full responsibility for the correctness, originality, and quality of all submitted work.